Building TerraStudio with AI

Afroze Amjad

Associate Director Engineering

What This Session Covers

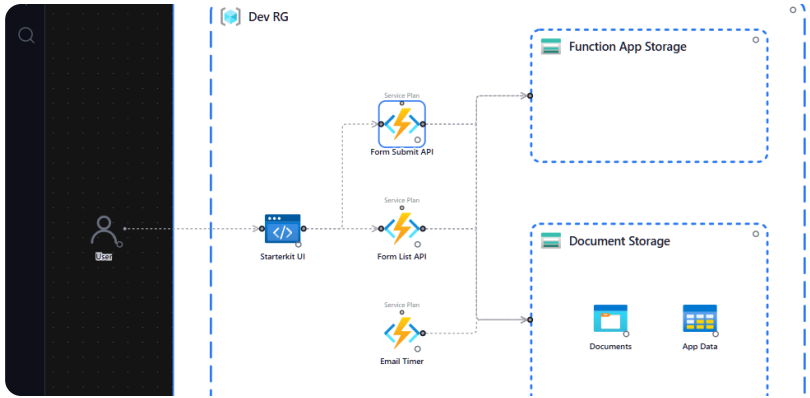
- Can one developer + AI build a production-grade desktop app from scratch?
- What breaks when AI writes 80% of the code?
- Hard numbers: velocity, bugs, cost, and the "fix tax"
- Key Takeaways

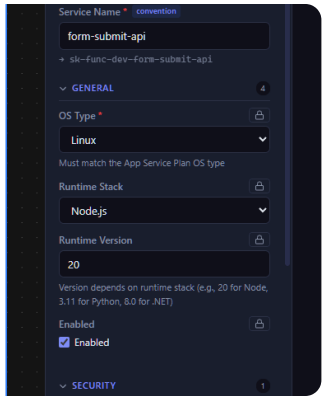
The Use Case

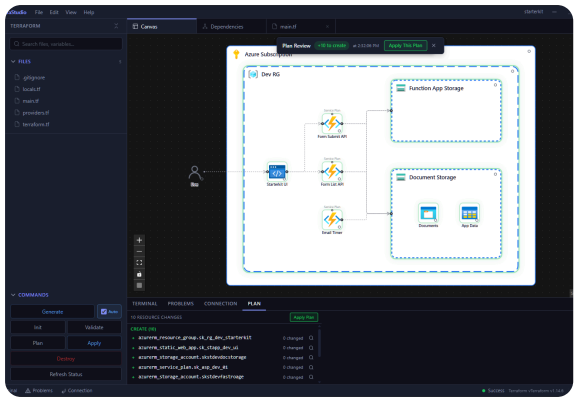
Build a full desktop infrastructure-as-code visual editor + CLI from zero

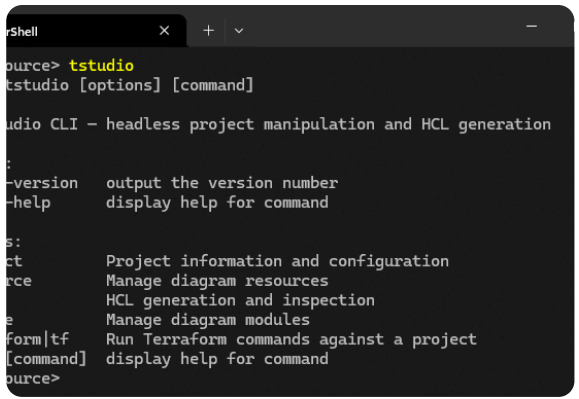
- **Visual canvas** for designing Azure/AWS architectures with drag-and-drop
- **Terraform HCL generation**, execution, and plan visualization
- 72+ cloud resources across Azure (50+), AWS (14+), and Azure AI (8)
- Cost estimation, modules, **i18n** (6 languages), **accessibility** (WCAG AA)
- **CLI** with HCL generation, project wizard, and native binary packaging

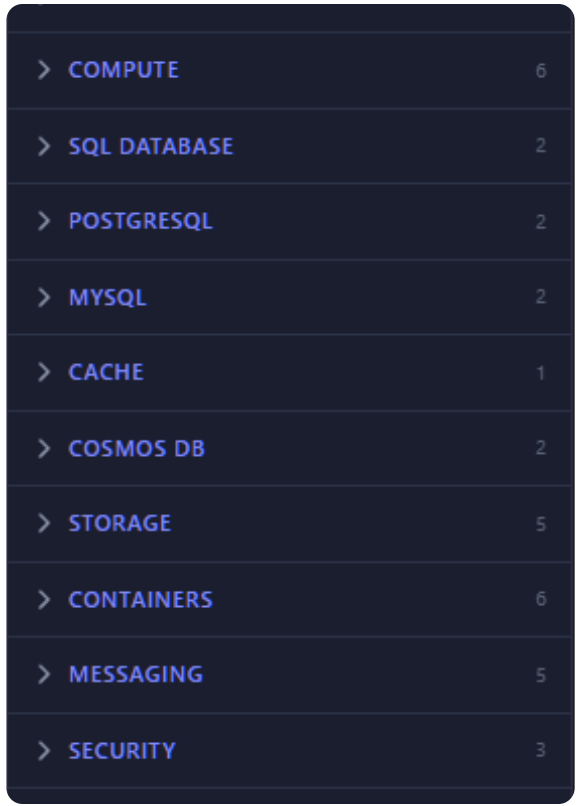
The Use Case


Dev RG
Function App Storage
Service Plan
Form Submit API
Service Plan
Form List API
Service Plan
Email Timer
Starterkit UI
User
Document Storage
Documents
App Data


Service Name * convention
form-submit-api
→ sk-func-dev-form-submit-api
GENERAL 4
OS Type *
Linux
Must match the App Service Plan OS type
Runtime Stack
Node.js
Runtime Version
20
Version depends on runtime stack (e.g., 20 for Node, 3.11 for Python, 8.0 for .NET)
Enabled
Enabled
SECURITY 1

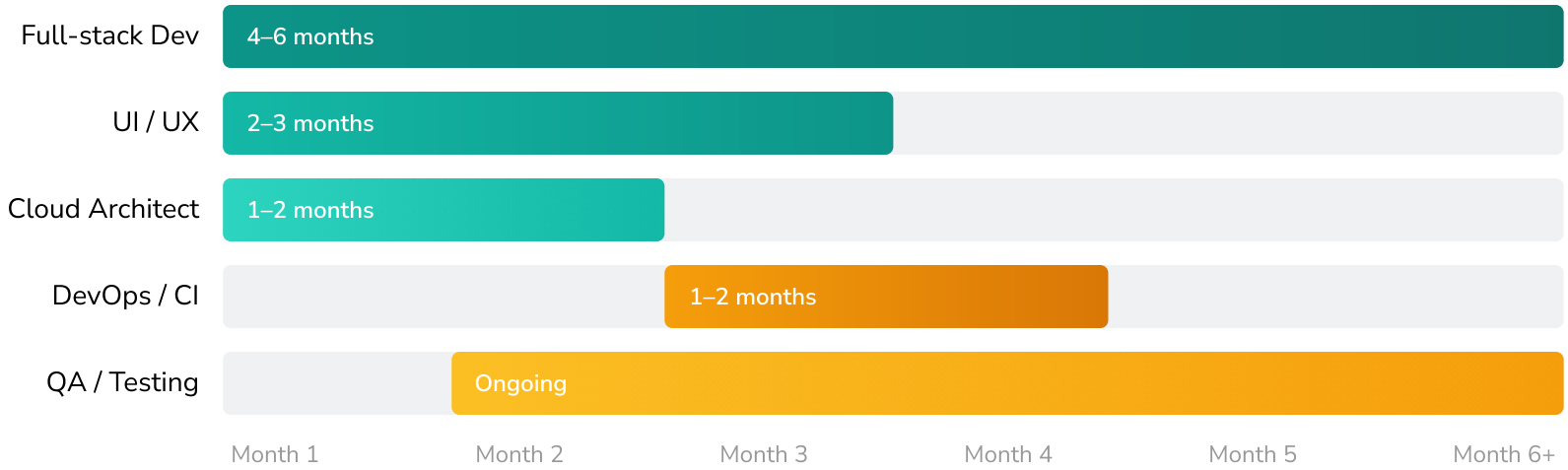

Dev RG
Function App Storage
Document Storage
Documents
App Data


rShell
ource> tstudio
tstudio [options] [command]
udio CLI – headless project manipulation and HCL generation
-version output the version number
-help display help for command
s:
ct Project information and configuration
rce Manage diagram resources
HCL generation and inspection
e Manage diagram modules
form|tf Run Terraform commands against a project
[command] display help for command
ource>


COMPUTE 6
SQL DATABASE 2
POSTGRESQL 2
MYSQL 2
CACHE 1
COSMOS DB 2
STORAGE 5
CONTAINERS 6
MESSAGING 5
SECURITY 3

The Traditional Way

Building this without AI - estimated timeline for a small team:



We did it in 33 days

The Journey: 33 Days, 5 Weeks

Week 1
Monorepo scaffold, 48-commit marathon, containers, undo/redo, theming, 20+ Azure resources

Week 2
Cost estimation, MCP server, multi-window, modules, templates

Week 3
AWS plugin (14 resources), i18n, accessibility, dependency graph

Week 4
Naming system collapse (7 fixes), CLI extraction, release pipeline hell (21 fixes)

Week 5
Resource audit (30 issues found), Azure AI plugin, CLI wizard

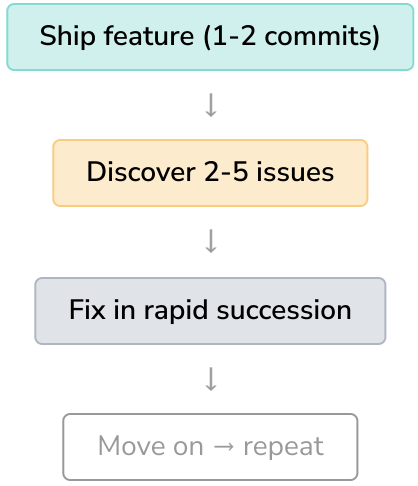
Weeks 1-3: 28% fix rate. **Weeks 4-5**: 47% fix rate. **48 versions** shipped.

The Journey: Interactive Timeline

Open Interactive Timeline

The Ship-Then-Fix Pattern

Every major feature followed this cadence - 16 instances across the project:



The Fix Chains

Every major subsystem had its own debugging saga - 48 total fix commits across 6 chains:

npm Publish Pipeline	OIDC, provenance, scoped packages, release-please - 7 patch versions, zero features	21 fixes
Naming Conventions	Slug → name → slug circular dependency. Each fix broke a different direction	7 fixes
Module System	Synthetic nodes leaked into HCL gen, export, save/load, and cost estimation	6 fixes
Container Model	canContain → canBeChildOf inversion cascaded through every container resource	5 fixes
TypeScript Types	Adding one reactive interface created implicit-any errors across the entire codebase	5 fixes
Drag-and-Drop	WebView2 ≠ browser - the most basic interaction took 4 attempts to get working	4 fixes

Where AI Failed - Every Time

Platform Blind Spots	WebView2 DnD, Svelte 5 proxies, CJS/ESM	12+ fixes
Wrong Architecture	canContain → canBeChildOf, naming cycles	9 rewrites
No Security Instinct	HCL injection shipped for 12 versions	Retrofitted
Cascading Blindness	Module nodes leaked into every pipeline	6 fixes
No Testing Initiative	3 test commits out of 338 (0.9%)	30 audit issues
CI/CD Incompetence	npm publish saga	21 fixes

AI chose the wrong architecture 9 out of 9 times when left to its own judgment.

Before and After

	With AI	Without AI
Time to product	33 days	4-6 months
Human hours	~80-100 hrs*	~2,000-3,000 hrs
Features shipped	48 versions, 72+ resources	Same scope
Bug rate	33% fix commits	~10-15%
Test coverage	0.9%	40-70%
Arch rewrites	9	1-2

Takeaway 1: AI Is a Brilliant Junior Dev

AI produces incredible velocity but requires a human to catch every:

Architectural mistakes	9 wrong abstractions - wrong every time when left to its own judgment
Security vulnerabilities	HCL injection shipped undetected for 12 versions
Platform quirks	WebView2, CJS/ESM, npm OIDC - blind spots in every runtime
Cascading failures	Module nodes, naming cycles - fixes locally, breaks globally

Weeks 1-3
28% fixes

→

Weeks 4-5
47% fixes

Velocity erodes when you move from building to hardening.

Takeaway 2: Architecture Is the New Bottleneck

The optimal approach is architect-led, AI-accelerated development:

Human	Design the architecture, security model, and CI pipeline
AI	Implement features at 10x speed
CI	Enforce tests and linting automatically
Human	Review before merge
Audit	Catch accumulated quality debt periodically

> The 33% fix tax is the price of velocity - and it compounds as complexity grows. The question isn't whether to pay it - it's whether your architecture and testing strategy can keep it from becoming the dominant cost.

Questions?

TerraStudio

github.com/afroze9/terrastudio

This Analysis

github.com/afroze9/terrastudio-devhistory